PYTHON OPERATORS THE COMPLETE TEACHING HANDBOOK

A Comprehensive, Line-by-Line Reference Guide

From Basic Arithmetic to Operator Overloading: Precedence, Evaluation, Truth Tables and Practice

Kajola Gbenga
Data Scientist & AI Engineer

Table of Contents

1. Introduction to Python Operators

1.1 What Are Operators?

An operator is a special symbol or keyword in Python that tells the interpreter to carry out a specific mathematical, logical, or comparison operation on one or more values. Operators are the verbs of a programming language: they take values and do something with them, producing a new value as a result. When you write 3 + 5, the + symbol is the operator, and it instructs Python to add the two numbers together, producing 8.

Every operator in Python is really shorthand for a more explicit operation happening behind the scenes. When you write a + b, Python does not perform some kind of built-in magic addition; instead it calls a special method on the object a, asking it to add itself to b. This connection between operators and methods becomes very important later in this handbook, in the section on operator overloading (section 14), where you will see that operators are simply a convenient syntax layered on top of ordinary method calls.

1.2 What Are Operands?

An operand is a value that an operator acts upon. In the expression 3 + 5, the numbers 3 and 5 are the operands, and + is the operator that combines them. Operands can be literal values (like 3 or 'hello'), variables (like x or total), or even entire expressions (like (a + b) in (a + b) * c, where the parenthesized expression as a whole becomes a single operand for the multiplication).

1.3 Operators vs Operands

Code Line	Explanation
`Operator`	The symbol or keyword that specifies the action to perform, for example +, ==, and, or is.
`Operand`	The value or values that the operator acts upon, for example the 3 and 5 in 3 + 5, or the single value x in the unary expression -x.
`Expression`	A combination of operators and operands that Python can evaluate down to a single resulting value, for example 3 + 5 * 2 evaluates to 13.

A useful analogy: in ordinary arithmetic language, if you say 'add 3 and 5', the word 'add' plays the role of the operator, and '3' and '5' play the role of operands. Python's operators work the same way, just expressed with symbols instead of words for most arithmetic and comparison operations, and with actual English keywords (and, or, not, in, is) for several of the others.

1.4 What Is an Expression?

An expression is any piece of code that Python can evaluate to produce a value. A single literal like 42 is the simplest possible expression, since it evaluates to itself. Combining operators and operands, like 42 + 8, or x > 10, or 'a' in my_list, are all expressions too, because each one evaluates down to a single value: a number, a string, or a boolean.

This is different from a statement, which is a complete instruction that Python executes but which does not necessarily produce a usable value on its own, such as an if statement, a for loop, or an assignment. It is common to build a statement out of one or more expressions; for example, the assignment statement total = price * quantity

contains the expression price * quantity, which is evaluated first, with its resulting value then stored into the variable total.

1.5 How Python Evaluates Expressions

When Python encounters an expression, it does not read and execute it strictly left to right the way a human might read a sentence. Instead, it builds an internal tree structure (technically an abstract syntax tree) that reflects operator precedence and associativity, then evaluates that tree from the innermost, highest-priority operations outward. In practice, this means Python decides which operator 'binds tightest' to its neighboring operands based on a fixed set of precedence rules (covered fully in section 10), and evaluates accordingly, rather than in the plain left-to-right order the code happens to be typed in.

```
result = 2 + 3 * 4
print(result)
```

Code Line	Explanation
`2 + 3 * 4`	Python does not compute this as (2 + 3) * 4; multiplication has higher precedence than addition, so Python evaluates 3 * 4 first (getting 12), and only then adds 2, producing 14.
`print(result)`	Displays the final evaluated value, 14, confirming that multiplication was applied before addition.

1.6 Unary vs Binary Operators

Operators are classified by how many operands they require. A unary operator acts on exactly one operand, for example the minus sign in -x, which simply negates the single value x. A binary operator acts on exactly two operands, for example the plus sign in x + y, which combines two separate values. Python also has one ternary (ternary meaning 'three-part') construct, the conditional expression a if condition else b, though this is technically an expression form rather than an operator symbol.

```
x = 5
print(-x)
print(x + 3)
```

Code Line	Explanation
`-x`	A unary operator: the minus sign here takes only one operand (x) and produces its negation, -5. This is not subtraction; there is nothing being subtracted from.
`x + 3`	A binary operator: the plus sign takes two operands, x and 3, and combines them, producing 8.

Important: The same symbol can be both a unary and a binary operator depending on context. The minus sign in -x is unary (negation), while the minus sign in x - y is binary (subtraction). Python tells these apart from the surrounding syntax, not from the symbol alone.

1.7 Operator Precedence and Associativity (Preview)

Operator precedence determines which operator is evaluated first when an expression contains more than one, similar to the familiar 'multiplication before addition' rule from ordinary arithmetic. Associativity determines the

order in which operators of the same precedence level are evaluated, typically left to right, though a few operators (like exponentiation) associate right to left instead. Both concepts are covered in full depth, with a complete precedence table, in section 10; they are introduced briefly here because they underlie how every expression in this handbook is evaluated.

2. Arithmetic Operators

2.1 Overview

Arithmetic operators perform mathematical calculations on numeric operands. Python supports seven arithmetic operators, summarized below before each is covered individually in full detail.

Code Line	Explanation
`+`	Addition: adds two operands together.
`-`	Subtraction: subtracts the right operand from the left.
`*`	Multiplication: multiplies two operands.
`/`	True division: divides the left operand by the right, always returning a float.
`//`	Floor division: divides and rounds the result down to the nearest whole number.
`%`	Modulus: returns the remainder of dividing the left operand by the right.
`**`	Exponentiation: raises the left operand to the power of the right.

2.2 Addition (+)

```
print(7 + 3)
print(2.5 + 1.5)
print('Hello, ' + 'World!')
print([1, 2] + [3, 4])
```

Code Line	Explanation
`7 + 3`	Adds two integers, producing the integer 10.
`2.5 + 1.5`	Adds two floats, producing the float 4.0.
`'Hello, ' + 'World!'`	When both operands are strings, + performs concatenation (joining), not addition in the numeric sense, producing 'Hello, World!'.
`[1, 2] + [3, 4]`	When both operands are lists, + concatenates them into a new list, producing [1, 2, 3, 4]; the original lists are left unchanged.

Real-world application: summing item prices in a shopping cart, accumulating a running total inside a loop, or joining two pieces of text such as a first and last name.

> **Common Mistake:** Attempting to add operands of incompatible types, such as 'age: ' + 25, raises a TypeError, because Python does not automatically convert a number into text (or vice versa) inside a + expression. Use str(25) to convert first, or an f-string, if the goal is to build a combined text message.

2.3 Subtraction (-)

```
print(10 - 4)
print(3.5 - 5)
print({1, 2, 3} - {2, 3})
```

Code Line	Explanation
`10 - 4`	Subtracts 4 from 10, producing 6.
`3.5 - 5`	Subtracts an integer from a float; Python automatically promotes the integer to a float for the calculation, producing -1.5.

Code Line	Explanation
`{1, 2, 3} - {2, 3}`	When both operands are sets, - computes the set difference: every element in the left set that is not also in the right set, producing {1}.

Unlike +, the minus sign has no special meaning for strings or lists; attempting 'abc' - 'a' raises a TypeError, since there is no defined notion of 'subtracting' one string from another.

2.4 Multiplication (*)

```
print(6 * 7)
print(2.0 * 3)
print('ab' * 3)
print([0] * 5)
```

Code Line	Explanation
`6 * 7`	Multiplies two integers, producing 42.
`2.0 * 3`	Multiplies a float by an integer, producing the float 6.0.
`'ab' * 3`	When one operand is a string and the other is an integer, * performs repetition, repeating the string that many times, producing 'ababab'.
`[0] * 5`	The same repetition behavior applies to lists and tuples, producing [0, 0, 0, 0, 0]; a common, fast idiom for pre-allocating a list of a fixed size.

Edge Case: [0] * 5 works well for immutable elements like numbers, but [[]] * 3 creates three references to the very same inner list rather than three independent empty lists, which is a frequent source of subtle bugs when the inner list is later mutated.

2.5 True Division (/)

```
print(7 / 2)
print(8 / 4)
print(10 / 0)
```

Code Line	Explanation
`7 / 2`	Divides 7 by 2 and always returns a float, producing 3.5, even though 7 and 2 are both integers.
`8 / 4`	Even when the division is exact, / still returns a float: 2.0, not 2. This is a deliberate design choice in Python 3, distinguishing true division from floor division.
`10 / 0`	Dividing by zero raises a ZeroDivisionError; Python does not return infinity or NaN for ordinary numeric division by zero the way some other languages or float-specific operations do.

2.6 Floor Division (//)

```
print(7 // 2)
print(-7 // 2)
print(7.5 // 2)
```

Code Line	Explanation
`7 // 2`	Divides 7 by 2 and rounds the result down to the nearest whole number, producing 3 (not 3.5).

Code Line	Explanation
`-7 // 2`	Floor division always rounds toward negative infinity, not toward zero; -7 divided by 2 is -3.5, and rounding that down (toward negative infinity) gives -4, not -3. This surprises many learners coming from languages where integer division truncates toward zero.
`7.5 // 2`	Floor division works on floats too, and still returns a float when either operand is a float; the result here is 3.0.

Real-world application: splitting a list of items into equally sized full batches, computing how many whole hours fit into a number of minutes, or converting a total number of seconds into whole minutes.

2.7 Modulus (%)

```
print(7 % 2)
print(-7 % 2)
print(10 % 3)
```

Code Line	Explanation
`7 % 2`	Returns the remainder after dividing 7 by 2, producing 1.
`-7 % 2`	Python's modulus result always takes the sign of the divisor (the right operand), so -7 % 2 produces 1, not -1, because it is defined so that (a // b) * b + (a % b) always equals a exactly.
`10 % 3`	Produces 1, since 3 goes into 10 three times with 1 left over.

Real-world application: determining whether a number is even or odd (n % 2 == 0), cycling through a fixed set of positions (index % length), or formatting a running count into groups (like inserting a line break every 10 items with count % 10 == 0).

2.8 Exponentiation (**)

```
print(2 ** 10)
print(9 ** 0.5)
print(2 ** -1)
```

Code Line	Explanation
`2 ** 10`	Raises 2 to the power of 10, producing 1024.
`9 ** 0.5`	A fractional exponent computes a root; raising 9 to the power of 0.5 computes its square root, producing 3.0.
`2 ** -1`	A negative exponent computes a reciprocal; 2 to the power of -1 is 1/2, producing 0.5.

Real-world application: compound interest calculations (principal * (1 + rate) ** years), computing areas and volumes, and any formula involving powers or roots.

2.9 Arithmetic Operators Across Data Types

Code Line	Explanation
`int + int`	Returns int. Example: 3 + 4 -> 7.
`int + float`	Returns float; Python promotes the int to a float first. Example: 3 + 4.0 -> 7.0.
`complex + int/float`	Returns complex. Example: (1+2j) + 3 -> (4+2j).
`str + str`	Concatenation, not numeric addition. Example: 'a' + 'b' -> 'ab'.
`str * int`	Repetition. Example: 'x' * 3 -> 'xxx'.

Code Line	Explanation
`list + list / list * int`	Concatenation / repetition, producing a new list.
`str + int`	Raises TypeError; Python never implicitly converts between text and numbers in arithmetic operators.

2.10 Common Mistakes and Edge Cases Summary

- Confusing / and // and getting an unexpectedly precise (or imprecise) result.
- Forgetting that floor division rounds toward negative infinity, not toward zero, for negative numbers.
- Dividing by zero with / or // or %, raising ZeroDivisionError.
- Trying to add a string and a number directly instead of converting one to match the other with str() or int()/float().
- Assuming floating-point arithmetic is always exact; 0.1 + 0.2 evaluates to 0.30000000000000004 due to how binary floating-point numbers represent decimal fractions, not a Python bug.

3. Comparison Operators

3.1 Overview

Comparison operators (also called relational operators) compare two operands and always produce a boolean result, either True or False. They are the building blocks of every conditional check in Python.

Code Line	Explanation
==	Equal to: True if both operands have equal value.
!=	Not equal to: True if the operands do not have equal value.
>	Greater than: True if the left operand is larger.
<	Less than: True if the left operand is smaller.
>=	Greater than or equal to.
<=	Less than or equal to.

Common Mistake: A very common beginner mistake is writing x = 5 (assignment) when the intention was x == 5 (comparison). A single = always assigns a value and never compares; a double == always compares and never assigns.

3.2 Comparing Numbers

```
print(5 == 5.0)
print(5 != 3)
print(3 < 10)
print(7 >= 7)
```

Code Line	Explanation
5 == 5.0	Produces True; Python compares numeric value, not type, so an int and a float that represent the same quantity are considered equal.
5 != 3	Produces True, since the two values differ.
3 < 10	Produces True.
7 >= 7	Produces True; >= includes equality.

3.3 Comparing Strings

```
print('apple' == 'apple')
print('apple' < 'banana')
print('Apple' == 'apple')
print('10' < '9')
```

Code Line	Explanation
'apple' == 'apple'	Produces True; string equality compares every character in sequence.
'apple' < 'banana'	String comparison is lexicographic (dictionary order), comparing character by character using each character's underlying Unicode code point; 'a' comes before 'b', so 'apple' is considered less than 'banana', producing True.
'Apple' == 'apple'	Produces False; string comparison is case-sensitive by default, and uppercase 'A' has a different Unicode code point than lowercase 'a'.
'10' < '9'	Produces True, which surprises many learners: this is a string

Code Line	Explanation
	comparison, not a numeric one, and the character '1' has a lower code point than the character '9', so '10' sorts before '9' alphabetically even though 10 is numerically larger.

3.4 Comparing Lists and Tuples

```
print([1, 2, 3] == [1, 2, 3])
print([1, 2, 3] < [1, 2, 4])
print([1, 2] < [1, 2, 3])
print((1, 2) == (1, 2))
```

Code Line	Explanation
`[1, 2, 3] == [1, 2, 3]`	Produces True; two lists are equal if they contain the same elements in the same order, not just if they occupy the same memory location.
`[1, 2, 3] < [1, 2, 4]`	Lists compare element by element, like strings; Python compares the first differing pair of elements (3 vs 4 here), producing True since 3 < 4.
`[1, 2] < [1, 2, 3]`	When one list is a prefix of the other, the shorter list is considered smaller, producing True.
`(1, 2) == (1, 2)`	Tuples compare the same way as lists, element by element, producing True.

3.5 Comparing Dictionaries and Sets

```
print({'a': 1, 'b': 2} == {'b': 2, 'a': 1})
print({1, 2, 3} == {3, 2, 1})
print({1, 2} < {1, 2, 3})
```

Code Line	Explanation
`{'a': 1, 'b': 2} == {'b': 2, 'a': 1}`	Produces True; dictionary equality checks that both dictionaries have exactly the same key-value pairs, regardless of the order the keys were inserted in.
`{1, 2, 3} == {3, 2, 1}`	Produces True; set equality checks membership only, since sets are inherently unordered.
`{1, 2} < {1, 2, 3}`	For sets, < means 'is a proper subset of', not a lexicographic comparison; this produces True because every element of the left set is contained in the right set, and the right set has additional elements.

Edge Case: Dictionaries do not support > or < comparison directly (only == and !=); attempting {'a': 1} < {'b': 2} raises a TypeError, since there is no universally sensible notion of one dictionary being 'less than' another.

3.6 Comparing Boolean Values

```
print(True == 1)
print(False == 0)
print(True > False)
```

Booleans are technically a subtype of int in Python, with True behaving as 1 and False behaving as 0 in any numeric or comparison context. This is why True == 1 and False == 0 both produce True, and True > False produces True as well, since 1 > 0.

3.7 Comparing Different Data Types

```
print(5 == '5')
print(5 == 5.0)
print([1, 2] == (1, 2))
```

Code Line	Explanation
`5 == '5'`	Produces False; an integer and a string are never considered equal by ==, no matter how similar they look, because they are fundamentally different types with no automatic conversion.
`5 == 5.0`	Produces True; numeric types (int, float, complex, bool) are compared by mathematical value across types, which is an intentional exception to the general 'different types are unequal' rule.
`[1, 2] == (1, 2)`	Produces False; a list and a tuple are never equal to each other via ==, even with identical elements in the same order, because Python considers the type itself part of what is being compared for these container types.

Attempting an ordering comparison (like <) between fundamentally incompatible types, such as 5 < 'hello', raises a TypeError in Python 3; unlike Python 2, Python 3 does not attempt to guess a sensible ordering between unrelated types.

4. Assignment Operators

4.1 The Basic Assignment Operator (=)

```
x = 10
name = 'Ada'
a, b = 1, 2
```

Code Line	Explanation
`x = 10`	Binds the name x to the value 10; = is not a mathematical statement of equality, it is an instruction to store a reference to the value on the right inside the variable named on the left.
`a, b = 1, 2`	Tuple/multiple assignment: unpacks the two values on the right and binds them to a and b respectively in a single line, equivalent to writing a = 1 followed by b = 2.

4.2 Augmented Assignment Operators

Augmented assignment operators combine an arithmetic or bitwise operation with assignment in a single, more concise step. Instead of writing x = x + 1, Python allows the shorthand x += 1, which reads and updates the variable in one operation.

Code Line	Explanation
`+=`	x += 5 is shorthand for x = x + 5.
`-=`	x -= 5 is shorthand for x = x - 5.
`*=`	x *= 5 is shorthand for x = x * 5.
`/=`	x /= 5 is shorthand for x = x / 5.
`//=`	x //= 5 is shorthand for x = x // 5.
`%=`	x %= 5 is shorthand for x = x % 5.
`**=`	x **= 5 is shorthand for x = x ** 5.
`&=`	x &= 5 is shorthand for x = x & 5 (bitwise AND, see section 8).
`|=`	x |= 5 is shorthand for x = x | 5 (bitwise OR, see section 8).
`^=`	x ^= 5 is shorthand for x = x ^ 5 (bitwise XOR, see section 8).
`>>=`	x >>= 2 is shorthand for x = x >> 2 (bitwise right shift, see section 8).
`<<=`	x <<= 2 is shorthand for x = x << 2 (bitwise left shift, see section 8).

```
total = 100
total += 50
print(total)

count = 10
count -= 3
print(count)

price = 20
price *= 1.15
print(price)
```

Code Line	Explanation
`total += 50`	Reads the current value of total (100), adds 50 to it, and stores the

Code Line	Explanation
	result (150) back into total, all in one step.
`count -= 3`	Reads count (10), subtracts 3, and stores the result (7) back into count.
`price *= 1.15`	Reads price (20), multiplies by 1.15 to apply a 15 percent increase, and stores the result (23.0) back into price.

4.3 Why Augmented Assignment Matters

- It is more concise and reads closer to natural language ('increase total by 50') than the fully spelled-out version.
- For mutable objects like lists, += can be more efficient than reassignment, since some augmented operations modify the object in place rather than creating an entirely new one.
- It is the standard, idiomatic way experienced Python programmers write accumulator and counter logic inside loops.

4.4 Augmented Assignment on Mutable Objects

```
numbers = [1, 2, 3]
numbers += [4, 5]
print(numbers)
```

Code Line	Explanation
`numbers += [4, 5]`	For a list, += calls the list's in-place extend behavior (via the __iadd__ special method), appending the new elements directly onto the existing list object rather than building an entirely new list and reassigning the name; the practical result, [1, 2, 3, 4, 5], looks the same as numbers = numbers + [4, 5], but is more memory-efficient and matters when other variables reference the same list object.
Edge Case: If another variable also points to the same list (for example other = numbers), an in-place += on numbers will be visible through other too, since both names refer to the same underlying object. A plain reassignment like numbers = numbers + [4, 5] would instead leave other pointing at the old, unchanged list. This distinction is a frequent source of confusion and bugs involving shared mutable state.	

4.5 Common Mistakes With Assignment Operators

- Using = when == was intended inside a condition, for example if x = 5: is actually a SyntaxError in Python (Python protects against this particular mistake, unlike some other languages).
- Chaining assignment across mutable default values, for example def f(x=[]): reuses the very same list object across every call that does not supply its own argument, a classic and surprising bug.
- Forgetting that augmented assignment still requires the variable to already exist; count += 1 raises a NameError if count was never previously assigned.
- Assuming a += b always behaves identically to a = a + b for every type; as shown above, this holds for immutable types like int, float and str, but can differ subtly for mutable types like list.

5. Logical Operators

5.1 Overview

Logical operators combine or invert boolean expressions. Python has exactly three: and, or, and not, written as plain English keywords rather than symbols, which is a deliberate readability choice distinguishing Python from languages that use && and ||.

Code Line	Explanation
`and`	True only if both operands are truthy.
`or`	True if at least one operand is truthy.
`not`	Inverts a single operand: True becomes False and vice versa.

5.2 Truth Values and Boolean Expressions

A boolean expression is any expression that Python treats as a truth value in a logical context. Every Python value has an inherent 'truthiness', not only the literal True and False; this is expanded on fully in section 12, but for the purposes of and/or/not, remember that Python evaluates the truthiness of whatever operand it is given, not just literal booleans.

5.3 Truth Tables

5.3.1 AND Truth Table

Code Line	Explanation
`True and True`	True
`True and False`	False
`False and True`	False
`False and False`	False

5.3.2 OR Truth Table

Code Line	Explanation
`True or True`	True
`True or False`	True
`False or True`	True
`False or False`	False

5.3.3 NOT Truth Table

Code Line	Explanation
`not True`	False
`not False`	True

5.4 Combining Multiple Conditions

```
age = 25
has_id = True
can_enter = age >= 18 and has_id
print(can_enter)
```

```
is_weekend = False
is_holiday = True
day_off = is_weekend or is_holiday
print(day_off)
```

Code Line	Explanation
`age >= 18 and has_id`	Evaluates the comparison age >= 18 first (producing True, since 25 >= 18), then combines it with has_id using and; since both operands are truthy, the whole expression is True.
`is_weekend or is_holiday`	Combines two booleans with or; since is_holiday is True, the whole expression evaluates to True even though is_weekend is False, because only one operand needs to be truthy.

5.5 Short-Circuit Evaluation (Preview)

Python's and and or do not always evaluate both operands. and stops and returns the left operand as soon as it finds a falsy value, since the overall result must be falsy regardless of the right operand. or stops and returns the left operand as soon as it finds a truthy value, since the overall result must be truthy regardless of the right side. This behavior, called short-circuit evaluation, is explored in complete depth with examples in section 11.

5.6 What and/or Actually Return

```
print(5 and 10)
print(0 and 10)
print(5 or 10)
print(0 or 10)
```

Code Line	Explanation
`5 and 10`	and does not always return True or False; it returns one of its actual operands. Since 5 is truthy, Python must check the second operand to determine the overall result, so it evaluates and returns 10, the last operand checked.
`0 and 10`	Since 0 is falsy, Python already knows the whole and expression must be falsy without checking the right side, so it short-circuits and returns 0 itself, the first falsy value found.
`5 or 10`	Since 5 is truthy, Python already knows the whole or expression must be truthy, so it short-circuits and returns 5 immediately, without evaluating 10 at all.
`0 or 10`	Since 0 is falsy, Python must check the next operand, so it evaluates and returns 10.

Practical Pattern: A very common and useful idiom exploits this behavior for default values: value = user_input or 'default' returns user_input if it is truthy (non-empty, non-zero, etc.), and falls back to 'default' only if user_input is falsy.

5.7 Common Mistakes With Logical Operators

- Using & and | (bitwise operators, see section 8) instead of and/or on plain booleans; this often happens to work by coincidence but is semantically wrong and behaves differently with non-boolean operands.

- Writing if x == 1 or 2: intending to check whether x equals 1 or 2, but this actually checks (x == 1) or 2, and since 2 is always truthy, the whole condition is always True regardless of x; the correct form is if x == 1 or x == 2: or if x in (1, 2):.
- Forgetting that not has lower precedence than comparison operators, so not x == y is evaluated as not (x == y), which is usually the intended meaning, but can confuse readers unfamiliar with the precedence table in section 10.

6. Identity Operators

6.1 Overview

Code Line	Explanation
`is`	True if both operands refer to the exact same object in memory.
`is not`	True if the operands refer to different objects in memory.

Every object created in Python lives at a specific location in memory, and has a unique identity for as long as it exists, retrievable with the built-in id() function. The is operator checks whether two names point to that same underlying object, not merely whether they hold equal values.

6.2 is vs ==

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)
print(a is b)
print(a is c)
```

Code Line	Explanation
`a == b`	Produces True; == compares value equality, and both lists contain the same elements in the same order.
`a is b`	Produces False; a and b are two separate list objects that happen to hold equal contents, occupying different locations in memory.
`a is c`	Produces True; c = a did not create a new list, it simply made c another name pointing at the exact same object that a already points to.

The Core Distinction: == asks 'do these two things have the same value?' while is asks 'are these two things literally the same object?' In everyday code, == is almost always the correct choice; is should be reserved for identity checks, most commonly comparing against None.

6.3 Comparing With None

```
result = None
if result is None:
    print('No result found')
```

The idiomatic and recommended way to check for None is always with is (or is not), never ==. This is because None is a singleton, meaning there is only ever one None object in an entire running Python program, so identity comparison is both semantically correct and slightly faster than a value comparison. Additionally, a poorly designed custom class could technically override == to behave unexpectedly, but it cannot change what is means, making is the more robust and universally reliable choice.

6.4 Small Integer and String Caching (A Common Trap)

```
x = 5
```

```
y = 5
print(x is y)

a = 500
b = 500
print(a is b)
```

Code Line	Explanation
`x is y`	Produces True. CPython (the standard Python implementation) pre-creates and caches small integers, roughly from -5 to 256, as shared singleton objects for efficiency, so any variable assigned the value 5 actually points to that same cached object.
`a is b`	This one is less predictable and often produces False, because 500 falls outside the small integer cache range, so a and b are typically two independently created integer objects that happen to hold equal values.
Warning: Relying on is to compare numbers (beyond None, True, and False) is considered bad practice and a bug waiting to happen, precisely because this caching behavior is an implementation detail of CPython, not a guarantee of the Python language itself, and can differ between Python versions or implementations. Always use == to compare numeric or string values.	

6.5 Identity With Booleans

```
flag = True
print(flag is True)
print(flag == True)
```

Both True and False are also singleton objects in Python, so flag is True is a perfectly reliable and valid check, and is generally the more idiomatic style over flag == True; however, in ordinary conditional code, simply writing if flag: is preferred over both, since it directly tests truthiness without an explicit comparison at all.

7. Membership Operators

7.1 Overview

Code Line	Explanation
`in`	True if the left operand is found within the right operand (a container).
`not in`	True if the left operand is not found within the right operand.

7.2 Membership in Strings

```
print('cat' in 'concatenate')
print('z' not in 'concatenate')
```

Code Line	Explanation
`'cat' in 'concatenate'`	Checks whether 'cat' appears anywhere as a contiguous substring within 'concatenate'; produces True, since the letters c-a-t appear in sequence inside the word.
`'z' not in 'concatenate'`	Produces True, since the letter 'z' does not appear anywhere in the string.

7.3 Membership in Lists and Tuples

```
fruits = ['apple', 'banana', 'cherry']
print('banana' in fruits)
print('mango' not in fruits)
```

For lists and tuples, in performs a linear search, checking each element in order until a match is found or the end is reached. This means membership checks on very large lists can be relatively slow (proportional to the list's length), which is one reason sets and dictionaries, both covered next, are often preferred when membership testing needs to happen frequently.

7.4 Membership in Sets

```
valid_codes = {'US', 'UK', 'NG', 'CA'}
print('NG' in valid_codes)
```

Membership testing on a set is, on average, dramatically faster than on a list, because sets are implemented internally using a hash table, allowing Python to jump almost directly to where an element would be stored rather than scanning every item one by one. When a program needs to repeatedly check whether values belong to a fixed collection, converting that collection to a set first is a common and effective performance optimization.

7.5 Membership in Dictionaries: Keys, Values and Items

```
student_grades = {'Ada': 95, 'Grace': 88, 'Alan': 76}
print('Ada' in student_grades)
print(95 in student_grades)
print(95 in student_grades.values())
print(('Ada', 95) in student_grades.items())
```

Code Line	Explanation
`'Ada' in student_grades`	By default, using in directly on a dictionary checks only its keys, producing True since 'Ada' is a key.
`95 in student_grades`	Produces False; 95 is a value, not a key, and plain in on a dictionary never searches values.
`95 in student_grades.values()`	.values() returns a view of just the dictionary's values, so checking membership against that view correctly finds 95, producing True.
`('Ada', 95) in student_grades.items()`	.items() returns a view of (key, value) tuple pairs; checking membership against it requires an exact matching tuple, producing True here since Ada does map to 95.

8. Bitwise Operators

8.1 Overview and Why They Exist

Bitwise operators work on the individual binary digits (bits) that make up an integer's underlying representation in memory, rather than treating the number as a single whole value the way arithmetic operators do. Every integer is stored internally as a sequence of bits, each either 0 or 1; bitwise operators let a program inspect, combine, or transform those bits directly. They are less commonly needed in everyday high-level Python code than arithmetic or comparison operators, but they are essential in systems programming, low-level data processing, working with flags and permission systems, graphics, networking protocols, and performance-critical numeric code.

Code Line	Explanation
`&`	Bitwise AND: sets each result bit to 1 only where both operand bits are 1.
`|`	Bitwise OR: sets each result bit to 1 where at least one operand bit is 1.
`^`	Bitwise XOR (exclusive or): sets each result bit to 1 where exactly one operand bit is 1, but not both.
`~`	Bitwise NOT: inverts every bit (unary operator, covered in detail in section 9).
`<<`	Left shift: shifts all bits left by the given number of positions, filling with zeros.
`>>`	Right shift: shifts all bits right by the given number of positions.

8.2 Binary Representation Refresher

A binary number is built from only two digits, 0 and 1, with each position representing an increasing power of 2 from right to left, exactly like each position in a decimal number represents an increasing power of 10. For example, the decimal number 13 is represented in binary as 1101, which breaks down as (1 * 8) + (1 * 4) + (0 * 2) + (1 * 1) = 8 + 4 + 0 + 1 = 13.

```
print(bin(13))
print(int('1101', 2))
```

Code Line	Explanation
`bin(13)`	The built-in bin() function converts a decimal integer into its binary string representation, prefixed with '0b'; the result here is '0b1101'.
`int('1101', 2)`	The built-in int() function, given a base of 2 as its second argument, parses a string of binary digits back into the equivalent decimal integer; the result here is 13.

8.3 Bitwise AND (&)

```
print(12 & 10)
```

12 in binary is 1100, and 10 in binary is 1010. Comparing bit by bit: position 1 (1 and 1 = 1), position 2 (1 and 0 = 0), position 3 (0 and 1 = 0), position 4 (0 and 0 = 0), giving 1000 in binary, which is 8 in decimal.

Code Line	Explanation
`1 & 1`	1
`1 & 0`	0
`0 & 1`	0
`0 & 0`	0

Practical application: & is commonly used together with a bitmask to check whether a specific flag bit is set within a combined integer, a pattern frequently used in low-level configuration flags and permission systems.

8.4 Bitwise OR (|)

```
print(12 | 10)
```

Using the same binary forms, 1100 and 1010: position 1 (1 or 1 = 1), position 2 (1 or 0 = 1), position 3 (0 or 1 = 1), position 4 (0 or 0 = 0), giving 1110 in binary, which is 14 in decimal.

Code Line	Explanation
`1 | 1`	1
`1 | 0`	1
`0 | 1`	1
`0 | 0`	0

Practical application: | is commonly used to combine multiple flag bits together into a single integer, turning several independent boolean settings on at once within one combined value.

8.5 Bitwise XOR (^)

```
print(12 ^ 10)
```

Using 1100 and 1010 again: position 1 (1 xor 1 = 0), position 2 (1 xor 0 = 1), position 3 (0 xor 1 = 1), position 4 (0 xor 0 = 0), giving 0110 in binary, which is 6 in decimal.

Code Line	Explanation
`1 ^ 1`	0
`1 ^ 0`	1
`0 ^ 1`	1
`0 ^ 0`	0

Practical application: XOR is frequently used to toggle specific bits (flipping a flag between on and off), to detect differences between two values, and in certain simple data encoding and checksum algorithms, since XOR-ing the same value twice always restores the original value: a ^ b ^ b always equals a.

8.6 Left Shift (<<) and Right Shift (>>)

```
print(3 << 2)
print(20 >> 2)
```

Code Line	Explanation
`3 << 2`	3 in binary is 11; shifting every bit two positions to the left gives 1100, which is 12; shifting left by n positions is mathematically equivalent to multiplying by 2 raised to the power of n, so 3 << 2

Code Line	Explanation
	equals 3 * (2 ** 2) = 12.
`20 >> 2`	20 in binary is 10100; shifting every bit two positions to the right gives 101, which is 5; shifting right by n positions is equivalent to floor-dividing by 2 raised to the power of n, so 20 >> 2 equals 20 // (2 ** 2) = 5.

Practical application: bit shifting is used for fast multiplication and division by powers of two, packing multiple small values into a single integer (for example combining a red, green and blue color channel into one 24-bit number), and low-level protocol or file-format parsing.

8.7 Flags and Masks: A Practical Example

```
READ = 0b0001
WRITE = 0b0010
EXECUTE = 0b0100

permissions = READ | WRITE
print(bin(permissions))

can_write = bool(permissions & WRITE)
print(can_write)

permissions_without_write = permissions & ~WRITE
print(bin(permissions_without_write))
```

Code Line	Explanation
`READ = 0b0001, ...`	Defines three independent permission flags, each occupying a single distinct bit position, using Python's 0b binary literal syntax.
`permissions = READ | WRITE`	Combines the READ and WRITE flags into one integer using bitwise OR, producing 0b0011; both permissions are now represented in a single value.
`bool(permissions & WRITE)`	Uses bitwise AND with the WRITE mask to isolate just that one bit; if the WRITE bit is set within permissions, the result is non-zero (truthy), so wrapping it in bool() cleanly answers 'does this include write access?'.
`permissions & ~WRITE`	~WRITE inverts every bit of WRITE (covered fully in section 9); ANDing that inverted mask against permissions clears (turns off) just the WRITE bit while leaving every other bit untouched, effectively removing a single flag from a combined set.

Real-World Context: This flags-and-masks pattern is exactly how many real systems represent file permissions (as in Unix's chmod), graphics rendering options, and network protocol headers, packing many independent boolean settings efficiently into a single integer.

9. Unary Operators

9.1 Overview

A unary operator acts on a single operand, as introduced in section 1.6. Python has four unary operators, each one already covered in its full binary or logical context earlier in this handbook; this section brings them together specifically to contrast their unary behavior.

Code Line	Explanation
`+`	Unary plus: returns the operand essentially unchanged.
`-`	Unary minus: returns the negation of the operand.
`~`	Bitwise NOT: inverts every bit of the operand.
`not`	Logical NOT: inverts the truthiness of the operand.

9.2 Unary Plus (+)

```
x = 5
print(+x)
y = -5
print(+y)
```

Unary plus is the least commonly useful of the four; for ordinary numbers it simply returns the same value unchanged, so +5 is 5 and +(-5) is -5. It exists mainly for symmetry with unary minus, and because custom classes can define a __pos__ method to give it meaningful custom behavior (see section 14 on operator overloading).

9.3 Unary Minus (-)

```
x = 5
print(-x)
y = -5
print(-y)
```

Code Line	Explanation
`-x`	Negates x, producing -5 from the positive value 5.
`-y`	Negates y, which is already -5, producing 5; negating a negative number produces a positive one.

Unary minus is genuinely common in everyday code: representing a temperature drop, a financial loss, a coordinate below an origin point, or reversing the direction of motion in a simulation.

9.4 Bitwise NOT (~)

```
print(~5)
print(~0)
print(~-1)
```

Bitwise NOT inverts every single bit of the operand's binary representation. Because Python integers are conceptually stored using two's complement representation with infinite precision, the mathematical result of ~x is always equal to -(x + 1), rather than a simple visible bit-flip of a fixed-width number.

Code Line	Explanation
`~5`	Equals -(5 + 1), producing -6.
`~0`	Equals -(0 + 1), producing -1.
`~-1`	Equals -(-1 + 1), producing 0.

Edge Case: Unlike languages with fixed-width integers (like 32-bit C integers), Python's ~ does not wrap around into some large positive number; it always follows the -(x + 1) rule cleanly regardless of the size of x, because Python integers have arbitrary precision.

9.5 Logical NOT (not)

```
print(not True)
print(not 0)
print(not '')
print(not 'hello')
```

Code Line	Explanation
`not True`	Produces False, the direct inversion of the literal boolean True.
`not 0`	Produces True; 0 is falsy (see section 12), so its logical negation is True.
`not ''`	Produces True; an empty string is falsy.
`not 'hello'`	Produces False; a non-empty string is truthy, so its negation is False.

Unlike ~ (which works bit by bit on the operand's binary representation and returns another number), not always evaluates the operand's overall truthiness first and always returns a plain boolean, True or False, regardless of the operand's original type.

9.6 Why Unary and Binary Versions of the Same Symbol Can Coexist

```
print(10 - 3)
print(10 - -3)
print(10 + -3)
```

Python's parser determines whether a + or - is unary or binary from its position in the expression: if it immediately follows another value or a closing parenthesis, it is treated as binary (an operation between two operands); if it appears at the start of an expression, right after an opening parenthesis, or right after another operator, it is treated as unary. 10 - -3 is read as 10 minus (negative 3), which equals 13; this is a common and legitimate pattern for expressing subtraction of a negative number.

10. Operator Precedence and Associativity

10.1 Why Precedence Matters

When an expression contains more than one operator, Python needs a consistent rule for deciding which operation to perform first, exactly the way ordinary arithmetic teaches 'multiplication before addition'. Operator precedence is this fixed hierarchy of rules; higher-precedence operators bind more tightly to their operands and are evaluated before lower-precedence ones. Without a shared, predictable set of rules, the same expression could be interpreted in different ways by different readers, or worse, silently produce a different answer than intended.

10.2 Complete Precedence Table (Highest to Lowest)

Code Line	Explanation
`( ) - Parentheses`	Grouping; always evaluated first, overriding every other rule below.
`** - Exponentiation`	Right-to-left associative (see 10.4); binds more tightly than unary minus on its left operand.
`+x, -x, ~x - Unary plus, minus, bitwise NOT`	Applied before most binary operators.
`*, /, //, % - Multiplication, division, floor division, modulus`	Left-to-right associative.
`+, - - Addition, subtraction`	Left-to-right associative.
`<<, >> - Bitwise shifts`	Left-to-right associative.
`& - Bitwise AND`	Left-to-right associative.
`^ - Bitwise XOR`	Left-to-right associative.
`| - Bitwise OR`	Left-to-right associative.
`==, !=, >, <, >=, <=, is, is not, in, not in - Comparisons`	Chain left-to-right (see section 19); all share the same precedence level.
`not`	Applied after comparisons, before and/or.
`and`	Evaluated before or.
`or`	Lowest precedence among the logical operators.
`if - else (conditional expression)`	Lower precedence than or.
`= and augmented assignment (+=, -=, etc.)`	Lowest precedence overall; always the very last thing evaluated in a statement.

Scope Note: This table is a practical teaching ordering of the operators covered in this handbook; the full CPython grammar has additional entries (like lambda, walrus :=, and comma-separated tuples) that fall outside this handbook's scope but occupy specific positions in the same overall hierarchy.

10.3 Step-by-Step Evaluation of a Complex Expression

```
result = 3 + 4 * 2 ** 2 - 1
print(result)
```

Code Line	Explanation
`Step 1: 2 ** 2`	Exponentiation has the highest precedence among the operators present, so it is evaluated first, producing 4. The expression is now conceptually 3 + 4 * 4 - 1.
`Step 2: 4 * 4`	Multiplication is evaluated next, since it outranks addition and

Code Line	Explanation
	subtraction, producing 16. The expression is now conceptually 3 + 16 - 1.
`Step 3: 3 + 16`	Addition and subtraction share the same precedence level, so Python evaluates them left to right; addition comes first here, producing 19.
`Step 4: 19 - 1`	The final subtraction is applied, producing the overall result, 18.

10.4 Associativity

Associativity determines the evaluation order among operators that share the exact same precedence level. Most binary operators in Python are left-associative, meaning a sequence of them is evaluated from left to right. Exponentiation (**) is a notable exception: it is right-associative, meaning a chain of ** operators groups from the right.

```
print(2 ** 3 ** 2)
print(10 - 3 - 2)
```

Code Line	Explanation
`2 ** 3 ** 2`	Because ** is right-associative, this is evaluated as 2 ** (3 ** 2), not (2 ** 3) ** 2. First 3 ** 2 is computed (9), then 2 ** 9 is computed, producing 512. Evaluating it left-associatively instead would have incorrectly produced 64.
`10 - 3 - 2`	Because - is left-associative, this is evaluated as (10 - 3) - 2, first producing 7, then 7 - 2, producing 5.

10.5 Using Parentheses to Clarify Intent

```
print((3 + 4) * 2)
print(3 + (4 * 2))
```

Parentheses always override the default precedence rules and are evaluated first, from the innermost pair outward. Beyond controlling evaluation order, parentheses are also valuable purely for readability: even when the default precedence would already produce the intended result, adding parentheses around a sub-expression can make the writer's intent immediately obvious to anyone reading the code later, including their future self.

10.6 Common Precedence Mistakes

- Assuming and and or have the same precedence; and is evaluated before or, so a or b and c is actually a or (b and c), not (a or b) and c.
- Forgetting that comparison operators outrank not, so not a == b is not (a == b), which is almost always the intended meaning, but can look ambiguous to a reader unfamiliar with the rule.
- Writing -3 ** 2 expecting 9, but unary minus has lower precedence than **, so this is actually -(3 ** 2), producing -9, not (-3) ** 2.
- Chaining bitwise operators with comparisons without parentheses, for example x & 1 == 1, which is actually parsed as x & (1 == 1) due to comparisons outranking &, almost never the intended meaning; the correct form is (x & 1) == 1.

11. Short-Circuit Evaluation

11.1 What Short-Circuit Evaluation Means

Short-circuit evaluation is the behavior where Python stops evaluating a logical expression as soon as the overall result is already determined, without bothering to evaluate the remaining operand(s). This was introduced briefly in section 5.5; this section explores it in full depth, including why it matters practically, not just what it does mechanically.

11.2 How and Short-Circuits

```
def check_a():
    print('Checking A')
    return False

def check_b():
    print('Checking B')
    return True

result = check_a() and check_b()
print(result)
```

Code Line	Explanation
`check_a() and check_b()`	Python calls check_a() first, which prints 'Checking A' and returns False. Since the left operand of and is already falsy, the overall result must be False no matter what check_b() would return, so Python never calls check_b() at all; 'Checking B' is never printed.
`print(result)`	Prints False, the value returned by check_a(), which and short-circuited to.

11.3 How or Short-Circuits

```
def check_a():
    print('Checking A')
    return True

def check_b():
    print('Checking B')
    return False

result = check_a() or check_b()
print(result)
```

Here check_a() returns True, so or already knows the overall result must be True regardless of check_b(); Python short-circuits and never calls check_b(), so 'Checking B' is never printed, and result is True.

11.4 Practical Applications of Short-Circuiting

11.4.1 Guard Conditions (Avoiding Errors)

```
data = None
if data is not None and len(data) > 0:
    print('Data available')
else:
    print('No data')
```

This is one of the most important practical uses of short-circuiting in everyday Python. If data is None, the left condition data is not None evaluates to False, and short-circuiting prevents Python from ever evaluating len(data), which would otherwise raise a TypeError since len() cannot be called on None. This 'guard condition' pattern, checking that a precondition holds before relying on it, is extremely common when working with data that might be missing or empty.

11.4.2 Avoiding Unnecessary Computation

```
def is_cached(key):
    print('Checking cache...')
    return key in cache

def expensive_calculation():
    print('Running expensive calculation...')
    return 42

result = is_cached('x') or expensive_calculation()
```

If is_cached('x') returns True, or short-circuits and expensive_calculation() is never called at all, saving whatever time or resources it would have consumed. This pattern, checking a cheap condition first and only falling back to an expensive one when necessary, is a common performance technique enabled directly by short-circuit evaluation.

11.5 Truthy/Falsy Recap in This Context

Short-circuit evaluation relies entirely on each operand's truthiness (covered fully in section 12), not strictly on literal True/False values. This is why data is not None and len(data) > 0 works correctly even though neither check_a() nor check_b() in the earlier examples returned a literal boolean in every case; and and or always work with whatever truthy or falsy value each operand actually produces.

11.6 Common Mistakes With Short-Circuit Evaluation

- Relying on a function call inside and/or for its side effects, assuming it will always run; if a preceding operand already determines the result, the function may never actually execute.
- Writing the guard condition in the wrong order, for example len(data) > 0 and data is not None, which fails to protect against data being None, since len(data) is evaluated first.
- Forgetting that and/or return one of their actual operands, not necessarily True/False, which can lead to unexpected non-boolean values being stored if the result is used directly, as shown in section 5.6.

12. Truthy and Falsy Values

12.1 The Concept of Truthiness

Every object in Python, not just the literal booleans True and False, has an implicit truth value when it is used in a boolean context, such as inside an if statement, a while loop condition, or as an operand to and, or, or not. An object is considered 'truthy' if Python treats it as equivalent to True in such a context, and 'falsy' if Python treats it as equivalent to False.

12.2 The Complete List of Falsy Values

Code Line	Explanation
`None`	The absence of a value is always falsy.
`False`	The literal boolean False.
`0`	The integer zero.
`0.0`	The float zero.
`0j`	The complex zero.
`'' (empty string)`	A string with zero characters.
`[] (empty list)`	A list with zero elements.
`() (empty tuple)`	A tuple with zero elements.
`{} (empty dict)`	A dictionary with zero key-value pairs.
`set() (empty set)`	A set with zero elements.
`range(0)`	An empty range object.

Every value not listed above, including any non-zero number, any non-empty string, any non-empty collection, and every user-defined object by default, is truthy.

12.3 Practical Examples

```
values = [None, False, 0, 0.0, '', [], (), {}, set(), 'hello', [1], 1, -1, ' ']
for v in values:
    print(repr(v), '->', bool(v))
```

Code Line	Explanation
`bool(None) / bool(False) / bool(0)`	All produce False, per the falsy list above.
`bool('') vs bool(' ')`	An empty string ('') is falsy, but a string containing only a single space (' ') is truthy, since it does technically contain one character; truthiness for strings is based purely on length (are there zero characters), not on whether the content looks meaningful to a human.
`bool([]) vs bool([1])`	An empty list is falsy; a list containing even one element, including an element that is itself falsy like [0], is truthy, since truthiness for containers is based on whether they have any elements, not on the truthiness of what is inside them.
`bool(-1)`	Produces True; any non-zero number, whether positive or negative, is truthy. Only exactly zero (in any numeric type) is falsy.

12.4 Using Truthiness Directly in Conditions

```
user_input = ''
if user_input:
    print(f'You entered: {user_input}')
else:
    print('No input provided')
```

Writing if user_input: directly, relying on truthiness, is considered more idiomatic ('Pythonic') style than the more verbose if user_input != '', provided the intent really is 'is this empty/missing/zero' rather than a strict comparison against one specific value. This idiom is extremely common for checking whether a variable holds meaningful data before using it.

12.5 Custom Objects and Truthiness

```
class ShoppingCart:
    def __init__(self, items):
        self.items = items
    def __bool__(self):
        return len(self.items) > 0

cart = ShoppingCart([])
if cart:
    print('Cart has items')
else:
    print('Cart is empty')
```

Code Line	Explanation
`def __bool__(self): ...`	Defining this special (dunder) method lets a custom class control its own truthiness, exactly like the built-in container types do; here a ShoppingCart is considered truthy only if it actually contains items.
`if cart:`	Because ShoppingCart defines __bool__, Python calls it automatically to determine truthiness; without a __bool__ (or a fallback __len__) method, every custom object instance would default to always being truthy, regardless of its contents.

How It Works Internally: If a class defines __len__ but not __bool__, Python falls back to using __len__ to determine truthiness: an object with a length of 0 is falsy, and any other length is truthy. This is exactly why built-in containers like lists and dictionaries, which define __len__, are automatically falsy when empty.

13. Operators Across Different Data Types

13.1 Why This Matters

The exact same operator symbol can behave in completely different, type-specific ways depending on what kind of operands it is given. This section consolidates that behavior across every major built-in type in one place, so the differences and boundaries are easy to see side by side, without duplicating the deep type-specific explanations already covered in your existing Data Types notes.

13.2 int and float

```
print(7 + 3)
print(7 / 3)
print(7 // 3)
print(7.0 == 7)
```

Arithmetic, comparison and bitwise operators (bitwise only for int, not float) all work as expected between numeric types, with Python automatically promoting an int to a float whenever the two are mixed in the same expression, as covered fully in section 2.

13.3 complex

```
z1 = 3 + 4j
z2 = 1 - 2j
print(z1 + z2)
print(z1 * z2)
print(z1 > z2)
```

Code Line	Explanation
`z1 + z2 / z1 * z2`	Addition and multiplication work on complex numbers following the standard rules of complex arithmetic, producing another complex number.
`z1 > z2`	Raises a TypeError; complex numbers support equality (==, !=) but have no defined ordering, since there is no universally agreed way to say one complex number is 'greater than' another the way real numbers can be ordered.

13.4 str

```
print('py' + 'thon')
print('ab' * 3)
print('a' < 'b')
print('py' - 'thon')
```

Code Line	Explanation
`'py' + 'thon'`	Concatenation, producing 'python'.
`'ab' * 3`	Repetition, producing 'ababab'.
`'a' < 'b'`	Lexicographic comparison based on Unicode code points, producing

Code Line	Explanation
	True (see section 3.3).
`'py' - 'thon'`	Raises a TypeError; strings do not support subtraction, since there is no built-in, unambiguous definition of what 'removing' one string from another would mean.

13.5 list

```
print([1, 2] + [3, 4])
print([1, 2] * 2)
print([1, 2, 3] == [1, 2, 3])
print([1, 2] - [1])
```

Code Line	Explanation
`[1, 2] + [3, 4]`	Concatenation, producing a new list [1, 2, 3, 4].
`[1, 2] * 2`	Repetition, producing [1, 2, 1, 2].
`[1, 2, 3] == [1, 2, 3]`	Element-wise equality comparison, producing True (see section 3.4).
`[1, 2] - [1]`	Raises a TypeError; unlike sets, lists do not support subtraction, because a list can contain duplicates and preserves order, so 'removing elements' is ambiguous without a specifically chosen method rather than an operator.

13.6 tuple

```
print((1, 2) + (3, 4))
print((1,) * 3)
print((1, 2) == (1, 2))
```

Tuples support the same + (concatenation) and * (repetition) behavior as lists, and the same element-wise comparison behavior, but because tuples are immutable, these operators always produce a brand new tuple rather than modifying anything in place; there is no augmented += that mutates a tuple the way there sometimes is for a list.

13.7 set

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

Code Line	Explanation
`a | b`	Set union: every element that appears in either set, producing {1, 2, 3, 4}.
`a & b`	Set intersection: only elements that appear in both sets, producing {2, 3}.
`a - b`	Set difference: elements in a that are not in b, producing {1}.
`a ^ b`	Set symmetric difference: elements in exactly one of the two sets but not both, producing {1, 4}.

Notice that sets reuse the same symbols as the bitwise operators (|, &, ^) from section 8, applying an analogous logical meaning at the level of set membership instead of individual bits; sets do not support + or * at all, since concatenation and repetition make no sense for an unordered collection with no duplicates.

13.8 dict

```
a = {'x': 1, 'y': 2}
b = {'y': 99, 'z': 3}
print(a | b)
print(a == {'x': 1, 'y': 2})
```

Code Line	Explanation
`a | b`	Dictionary union (available in Python 3.9 and later): merges two dictionaries into a new one; when a key exists in both, the value from the right-hand dictionary (b) wins, producing {'x': 1, 'y': 99, 'z': 3}.
`a == b`	Compares key-value pairs for equality, ignoring insertion order, as covered in section 3.5.

Dictionaries do not support +, -, *, or ordering comparisons (<, >) at all; only == , != , and (from 3.9 onward) | and |= are defined.

13.9 bool

```
print(True + True)
print(True + 1)
print(False * 10)
```

Because bool is a subtype of int (as noted in section 3.6), arithmetic operators work on booleans exactly as they would on 0 and 1; True + True produces 2 (an int, not a bool), and False * 10 produces 0. This is a real, occasionally useful technique: summing a list of booleans, for example sum(x > 0 for x in numbers), directly counts how many elements satisfy the condition, since each True contributes 1 to the total.

13.10 Cross-Type Operator Support Summary

Code Line	Explanation
`+ (numeric add / concatenation)`	int, float, complex, str, list, tuple. Not supported: dict, set, bool alone as a distinct type (falls back to int).
`* (numeric multiply / repetition)`	int, float, complex, str (with int), list (with int), tuple (with int). Not supported: dict, set.
`- (numeric subtract / set difference)`	int, float, complex, set. Not supported: str, list, tuple, dict.
`/, //, % (division family)`	int, float, complex (/ only; // and % unsupported for complex). Not supported: str, list, tuple, set, dict.
`&, |, ^ (bitwise / set ops)`	int (bitwise), set (set operations), dict (| only, merge). Not supported: str, list, tuple, float.
`==, != (equality)`	Every built-in type supports these.
`<, >, <=, >= (ordering)`	int, float, complex (== / != only, no ordering), str, list, tuple, set (subset/superset meaning), bool. Not supported: dict.

14. Operator Overloading (Dunder Methods)

14.1 What Operator Overloading Means

Operator overloading is the mechanism that lets a custom class define its own meaning for a built-in operator symbol. As introduced in section 1.1, every operator in Python is really a convenient syntax for calling a specific, predictably named special method (also called a 'dunder' method, short for 'double underscore', since each name is surrounded by two underscores on each side, like __add__). When you write a + b, Python does not have any built-in knowledge of what + means for your custom class; instead, it looks for and calls a.__add__(b) behind the scenes, and it is entirely up to your class's definition of __add__ to decide what happens.

This is a genuinely powerful feature: it is precisely why numpy arrays can be added element-wise with +, why pandas DataFrames can be compared with ==, and why your own custom classes, such as a Vector, Money, or Matrix class, can support natural, readable mathematical syntax instead of forcing every user to call verbose method names like vector.add(other_vector).

14.2 A Complete Worked Example: A Money Class

```
class Money:
    def __init__(self, amount, currency='USD'):
        self.amount = amount
        self.currency = currency

    def __repr__(self):
        return f'Money({self.amount}, {self.currency!r})'

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError('Cannot add different currencies')
        return Money(self.amount + other.amount, self.currency)

    def __sub__(self, other):
        return Money(self.amount - other.amount, self.currency)

    def __mul__(self, factor):
        return Money(self.amount * factor, self.currency)

    def __eq__(self, other):
        return self.amount == other.amount and self.currency == other.currency

    def __lt__(self, other):
        return self.amount < other.amount

wallet = Money(50) + Money(25)
print(wallet)
print(Money(50) < Money(75))
```

Code Line	Explanation
`def __add__(self, other): ...`	Called automatically whenever Money(50) + Money(25) is written;

Code Line	Explanation
	self is the left operand, other is the right operand. It validates that both amounts share the same currency, then returns a brand new Money object holding the summed amount.
`def __sub__(self, other): ...`	Defines the behavior for -, following the same pattern as __add__.
`def __mul__(self, factor): ...`	Defines the behavior for *, here allowing a Money object to be scaled by a plain number, useful for calculating something like a 10 percent discount with wallet * 0.9.
`def __eq__(self, other): ...`	Defines the behavior for ==; without this method, Python's default == would fall back to identity comparison (equivalent to is), meaning two separate Money(50) objects with identical amounts would incorrectly compare as not equal.
`def __lt__(self, other): ...`	Defines the behavior for <, which then also makes sorting a list of Money objects with sorted() work correctly out of the box.

14.3 Reference Table of Overloadable Operators

Code Line	Explanation
`__add__(self, other)`	Defines the behavior of + (addition / concatenation).
`__sub__(self, other)`	Defines the behavior of - (subtraction).
`__mul__(self, other)`	Defines the behavior of * (multiplication / repetition).
`__truediv__(self, other)`	Defines the behavior of / (true division).
`__floordiv__(self, other)`	Defines the behavior of // (floor division).
`__mod__(self, other)`	Defines the behavior of % (modulus).
`__pow__(self, other)`	Defines the behavior of ** (exponentiation).
`__eq__(self, other)`	Defines the behavior of == (equal to).
`__ne__(self, other)`	Defines the behavior of != (not equal to); Python auto-derives this from __eq__ if __ne__ is not explicitly defined.
`__lt__(self, other)`	Defines the behavior of < (less than).
`__le__(self, other)`	Defines the behavior of <= (less than or equal to).
`__gt__(self, other)`	Defines the behavior of > (greater than).
`__ge__(self, other)`	Defines the behavior of >= (greater than or equal to).
`__and__(self, other)`	Defines the behavior of & (bitwise AND).
`__or__(self, other)`	Defines the behavior of | (bitwise OR).
`__xor__(self, other)`	Defines the behavior of ^ (bitwise XOR).
`__invert__(self)`	Defines the behavior of ~ (bitwise NOT); note this takes only self, since ~ is unary.
`__lshift__(self, other)`	Defines the behavior of << (left shift).
`__rshift__(self, other)`	Defines the behavior of >> (right shift).
`__contains__(self, item)`	Defines the behavior of in / not in for this object as the right-hand operand.
`__getitem__(self, key)`	Defines the behavior of square-bracket indexing, obj[key], which is closely related to operator behavior and enables both indexing and, indirectly, iteration and membership testing.

14.4 Overloading Membership and Indexing

```
class TemperatureLog:
    def __init__(self, readings):
        self.readings = readings
```

```
    def __contains__(self, value):
        return value in self.readings

    def __getitem__(self, index):
        return self.readings[index]

log = TemperatureLog([21.5, 22.0, 19.8, 23.1])
print(23.1 in log)
print(log[0])
```

Code Line	Explanation
`23.1 in log`	Because TemperatureLog defines __contains__, Python calls log.__contains__(23.1) automatically whenever the in operator is used with log on the right-hand side, allowing a custom class to support the membership operators from section 7 in a fully natural way.
`log[0]`	Because TemperatureLog defines __getitem__, square-bracket indexing works directly on a log instance, delegating to the underlying self.readings list.

14.5 Why This Section Is Considered Advanced

- It requires a solid understanding of classes and methods (covered in your existing Functions and Methods notes) as a prerequisite.
- It requires understanding that operators are not magic; they are a thin, consistent syntactic layer over ordinary method calls, dispatched based on the type of the left (and sometimes right) operand.
- Getting it right requires careful attention to edge cases: what should __eq__ do when other is not a Money instance at all? What should __lt__ do for objects that cannot be meaningfully ordered? Well-designed overloaded operators handle these gracefully, typically by returning NotImplemented rather than raising an unexpected error, allowing Python to try the reverse operation or fall back to a sensible default.

15. Operators in Conditional Statements

15.1 The Role of Operators in if/elif/else

Every if, elif and else structure in Python is driven by an expression that operators build. The condition following if or elif is evaluated for its truthiness (section 12), and comparison, logical, membership and identity operators are what typically construct that condition from the surrounding variables and values.

15.2 A Single Condition

```
temperature = 35
if temperature > 30:
    print('It is hot today')
```

The comparison operator > builds the boolean expression that the if statement tests; if temperature > 30 evaluates to True, the indented block underneath runs.

15.3 elif and else With Multiple Conditions

```
score = 72
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
else:
    grade = 'F'
print(grade)
```

Code Line	Explanation
`if score >= 90: ...`	Python checks each condition from top to bottom, in order, and runs only the first block whose condition is True, then skips every remaining elif and else entirely.
`elif score >= 70:`	By the time this line is reached, Python has already confirmed score is not >= 90 and not >= 80 from the earlier failed checks, so this comparison only needs to distinguish the 70-79 range from what remains.
`else:`	Runs only if every preceding condition was False; it requires no operator of its own, since it is the catch-all fallback.

15.4 Nested Conditions

```
age = 20
has_ticket = True

if age >= 18:
    if has_ticket:
        print('Entry allowed')
```

```
    else:
        print('Need a ticket')
else:
    print('Must be 18 or older')
```

Nested conditions place one if statement entirely inside the block of another, useful when the second check only makes sense or is only relevant after the first has already passed. This same logic could often be flattened using logical and, as shown next, which is frequently more readable when the nested structure does not need genuinely different branches at each level.

15.5 Combining Multiple Conditions With Logical Operators

```
age = 20
has_ticket = True

if age >= 18 and has_ticket:
    print('Entry allowed')
else:
    print('Entry denied')
```

This flattens the nested version above into a single condition using and, which is often clearer when there is no need to give a different specific message for each individual failing condition.

15.6 Using in and is Inside Conditions

```
role = 'admin'
allowed_roles = ('admin', 'moderator', 'owner')

if role in allowed_roles:
    print('Access granted')

result = None
if result is None:
    print('No result yet')
```

Membership (in) and identity (is) operators are extremely common inside conditions, particularly for checking against a fixed set of allowed values, and for the idiomatic None check discussed in section 6.3.

16. Operators in Loops

16.1 Operators Driving while Loop Conditions

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Code Line	Explanation
`count < 5`	A comparison operator builds the condition that a while loop re-checks before every single iteration; as soon as this condition becomes False, the loop stops.
`count += 1`	The augmented assignment operator from section 4.2 updates the counter each time through the loop; without this line, count would never change, the condition would never become False, and the loop would run forever (an infinite loop).

16.2 Counters and Accumulators

```
numbers = [4, 8, 15, 16, 23, 42]
total = 0
count = 0
for n in numbers:
    total += n
    count += 1
average = total / count
print(average)
```

Code Line	Explanation
`total += n`	An accumulator pattern: total is progressively built up across every iteration of the loop using +=, ending with the sum of every number in the list.
`count += 1`	A counter pattern: count tracks exactly how many iterations have occurred, incrementing by 1 each time.
`total / count`	True division combines the two accumulated values at the end to compute the average, once the loop has finished.

16.3 Conditions Inside Loops (Filtering)

```
numbers = [4, 8, 15, 16, 23, 42]
evens = []
for n in numbers:
    if n % 2 == 0:
        evens.append(n)
print(evens)
```

Code Line	Explanation
`n % 2 == 0`	Combines the modulus operator (section 2.7) with an equality comparison to test whether each number is even; this pattern, filtering a collection based on a condition built from operators, is one of the single most common tasks in everyday programming.

16.4 Membership Operators Controlling Loop Behavior

```
blocked_users = {'spammer1', 'spammer2'}
usernames = ['ada', 'spammer1', 'grace', 'spammer2', 'alan']

active_users = []
for user in usernames:
    if user not in blocked_users:
        active_users.append(user)
print(active_users)
```

The membership operator not in filters out any username found in the blocked_users set, demonstrating how in/not in from section 7 combine naturally with a for loop to build a filtered result list.

16.5 Break, Continue, and Operators Together

```
numbers = [4, 8, 15, 16, 23, 42]
for n in numbers:
    if n > 20:
        break
    if n % 2 != 0:
        continue
    print(n)
```

Code Line	Explanation
`if n > 20: break`	As soon as a number greater than 20 is found, the comparison operator's result triggers break, which exits the loop immediately, skipping any remaining numbers entirely.
`if n % 2 != 0: continue`	For any odd number (where the remainder after dividing by 2 is not 0), continue skips the rest of the current iteration's body and moves straight to the next number, without executing the print(n) line below it.

17. Operators with Strings and Collections

17.1 String Concatenation Revisited

```
first = 'Ada'
last = 'Lovelace'
full_name = first + ' ' + last
print(full_name)
```

Chaining multiple + operators builds a longer string step by step; Python evaluates left to right (string concatenation is left-associative, per section 10.4), first combining first and ' ', then combining that result with last.

> **Performance Note:** Repeated + concatenation inside a loop over many strings is inefficient, because each + creates an entirely new string object, and strings are immutable, so the whole content is copied every single time. For joining many pieces, ".join(list_of_strings) is significantly faster, though .join() is a string method rather than an operator, so it falls outside this handbook's scope covered in your Methods notes.

17.2 String Repetition Revisited

```
border = '-' * 40
print(border)
print('=' * 20 + ' MENU ' + '=' * 20)
```

Repetition (*) combined with concatenation (+) is a common, simple way to build formatted text dividers, section headers, or progress indicators directly with operators, without needing any string formatting method.

17.3 Membership Across Sequence Types

```
text = 'the quick brown fox'
print('quick' in text)

words = text.split()
print('fox' in words)

print('o' in words)
```

Code Line	Explanation
`'quick' in text`	Substring membership within a string: checks for a contiguous sequence of characters.
`'fox' in words`	Element membership within a list: checks whether the exact string 'fox' is one of the list's elements, not a substring of any of them.
`'o' in words`	Produces False; even though the letter 'o' does appear inside the word 'fox' and 'brown', in on a list checks whole elements, not substrings within those elements; there is no element in the list that is exactly equal to 'o'.

17.4 Sequence Comparison Recap

```
print('abc' < 'abd')
print([1, 2, 3] < [1, 3])
print((1, 2) < (1, 2, 0))
```

Strings, lists and tuples all share the same comparison algorithm: compare elements pairwise from the start, and the first position where they differ decides the result; if one sequence is a complete prefix of the other, the shorter one is considered smaller. This unified rule, first introduced in sections 3.3 and 3.4, applies consistently across every ordered sequence type.

17.5 Set Operations as a Data-Cleaning Tool

```
required_columns = {'name', 'age', 'email'}
uploaded_columns = {'name', 'age', 'phone'}

missing = required_columns - uploaded_columns
extra = uploaded_columns - required_columns
shared = required_columns & uploaded_columns

print('Missing:', missing)
print('Extra:', extra)
print('Shared:', shared)
```

Code Line	Explanation
`required_columns - uploaded_columns`	Set difference identifies every required column that is absent from the uploaded data, here {'email'}.
`uploaded_columns - required_columns`	Set difference in the other direction identifies unexpected extra columns not on the required list, here {'phone'}.
`required_columns & uploaded_columns`	Set intersection identifies the columns present in both, here {'name', 'age'}.

This is a genuinely common real-world pattern: using set operators to compare two collections of column names, tags, user IDs, or permissions, quickly answering 'what's missing', 'what's extra', and 'what's shared' in three concise lines.

18. Common Operator Errors and How to Fix Them

18.1 TypeError From Incompatible Operands

```
age = 25
message = 'Age: ' + age
```

Code Line	Explanation
The error	TypeError: can only concatenate str (not "int") to str
Why it happens	+ requires both operands to be the same general kind for concatenation; Python does not implicitly convert an int into text within a + expression, unlike some looser-typed languages.
The fix	Convert the number explicitly first: 'Age: ' + str(age), or better, use an f-string: f'Age: {age}', which handles the conversion automatically and is generally the more readable, modern style.

18.2 ZeroDivisionError

```
total_score = 0
num_students = 0
average = total_score / num_students
```

Code Line	Explanation
The error	ZeroDivisionError: division by zero
Why it happens	/, //, and % all raise this error whenever the right operand is exactly zero, since division by zero is mathematically undefined.
The fix	Guard the division with a conditional check first, exploiting short-circuit evaluation from section 11: average = total_score / num_students if num_students > 0 else 0.

18.3 Incorrect Use of is

```
score = 100
if score is 100:
    print('Perfect score')
```

Code Line	Explanation
The problem	This may work by coincidence today, due to CPython's small-integer caching described in section 6.4, but it is not a reliable or portable check, and modern versions of Python even emit a SyntaxWarning against comparing a literal with is.
The fix	Always use == for value comparisons: if score == 100:. Reserve is strictly for identity checks, primarily against None, True, and False.

18.4 Incorrect Use of = (Assignment vs Comparison)

```
# if x = 5:      <- this line raises a SyntaxError in Python
x = 5
```

```
if x == 5:
    print('x is five')
```

Unlike some other languages, Python treats if x = 5: as an outright SyntaxError rather than silently allowing an accidental assignment inside a condition, which is a deliberate safety feature. The fix is simply to remember that comparison always requires the double equals sign, ==.

18.5 Incorrect Precedence Assumptions

```
discount = True
price = 100
final = price - 10 if discount else price
print(final)
```

Code Line	Explanation
A common mistake	Writing price - 10 if discount else price without realizing the conditional expression (if/else) has lower precedence than -, so this is actually evaluated as (price - 10) if discount else price, which happens to be the intended behavior here, but only by chance of how the expression was structured; a slightly different arrangement could easily produce a surprising result.
The safer fix	Use parentheses generously around any conditional expression mixed with arithmetic to make the intended grouping unambiguous: price - (10 if discount else 0).

18.6 Mixing Incompatible Data Types

```
total = 100
result = total + [1, 2, 3]
```

Code Line	Explanation
The error	TypeError: unsupported operand type(s) for +: 'int' and 'list'
Why it happens	Even though + is defined for both int and list individually, it is not defined between an int and a list; Python's operator dispatch (see section 14) only succeeds when at least one operand's type explicitly knows how to combine with the other's type.
The fix	Ensure both operands are a compatible, matching kind, for example converting or restructuring the data so the operation makes logical sense, such as summing the list first with sum([1, 2, 3]) before adding it to total.

18.7 Unexpected Boolean Results From Operator Precedence

```
x = 5
if not x == 5:
    print('Not five')
else:
    print('Is five')
```

This example actually works correctly (printing 'Is five'), because, as covered in section 10.6, not has lower precedence than ==, so it is parsed as not (x == 5). The risk is not that this specific example is wrong, but that a

reader unfamiliar with this precedence rule might misjudge the evaluation order in a more complex expression; adding explicit parentheses, not (x == 5), removes all ambiguity.

18.8 Chained Comparison Mistakes

```
x = 5
print(1 < x < 10)
print(1 < x and x < 10)

print(1 < x < 10 == True)
```

Chained comparisons have their own dedicated section (19) because their behavior, while convenient, is genuinely easy to misunderstand; the last line above is a classic trap, since it chains 1 < x, x < 10, and 10 == True together as three separate linked comparisons, not as (1 < x < 10) == True, which is a common but incorrect assumption.

19. Chained Comparisons

19.1 What a Chained Comparison Is

Python allows multiple comparison operators to be chained together in a single expression, closely mirroring how mathematical notation writes something like 1 < x < 10 to mean 'x is between 1 and 10'. This is a genuine, first-class language feature, not merely syntax sugar layered awkwardly on top of separate comparisons.

```
x = 5
print(1 < x < 10)
```

This reads naturally as 'is x between 1 and 10', and Python evaluates it exactly that way.

19.2 How Chained Comparisons Work Internally

A chained comparison a < b < c is evaluated by Python as the logical equivalent of (a < b) and (b < c), with one crucial difference: the middle value (b in this case) is only evaluated once, not twice, even though it conceptually participates in two separate comparisons. This matters when the middle expression involves a function call with side effects.

```
def get_value():
    print('Evaluating...')
    return 5

print(1 < get_value() < 10)
```

'Evaluating...' is printed only once, confirming that get_value() is called exactly one time, with its single returned value then compared against both 1 and 10, rather than being called separately for each half of the chain.

19.3 Chained Equality

```
a = 5
b = 5
c = 5
print(a == b == c)

d = 6
print(a == b == d)
```

Code Line	Explanation
`a == b == c`	Equivalent to (a == b) and (b == c); since all three variables hold 5, both underlying comparisons are True, so the whole chain is True.
`a == b == d`	a == b is True (both are 5), but b == d is False (5 does not equal 6), so the overall chained result is False, exactly matching what and would produce.

19.4 How Chained Comparisons Differ From Separate and Expressions

For simple variable comparisons, a < b < c and (a < b) and (b < c) produce identical results and read almost identically. The genuine practical difference emerges specifically when one of the compared expressions has a

side effect (like the get_value() function above) or is expensive to compute: the chained form guarantees that shared middle expression is evaluated only once, while writing out two fully separate and-linked comparisons would evaluate it twice, which could produce different results if the expression is not perfectly repeatable, or simply waste time recomputing it.

19.5 Mixing Different Comparison Operators in a Chain

```
x = 7
print(0 <= x < 10)
print(10 > x >= 5)
```

Chains are not limited to a single repeated operator; different comparison operators can be mixed freely within the same chain, as long as each adjacent pair makes logical sense, following the same 'each link ANDed together' rule described above.

19.6 A Genuine Trap: Chaining an Equality With a Boolean

```
x = 5
print(1 < x < 10 == True)
```

This produces False, which surprises many learners. It is evaluated as (1 < x) and (x < 10) and (10 == True); the first two parts are True, but the third part, 10 == True, is False (since True behaves as 1, not 10, per section 3.6), making the entire chain False. The lesson: never assume a chained comparison silently collapses down to a single overall boolean before being compared against another value; every additional == in the chain is a fully separate, equally weighted link.

20. Practical Real-World Applications

20.1 Sales Analysis

```
daily_sales = [1200, 950, 1800, 0, 2100, 1650, 300]

total_sales = sum(daily_sales)
average_sales = total_sales / len(daily_sales)
best_day = max(daily_sales)
days_above_average = [s for s in daily_sales if s > average_sales]
closed_days = [s for s in daily_sales if s == 0]

print(f'Total: {total_sales}, Average: {average_sales:.2f}')
print(f'Days above average: {len(days_above_average)}')
print(f'Closed days: {len(closed_days)}')
```

This combines arithmetic (sum and division for the average), comparison (> to find above-average days, == to detect a closed day recorded as 0), and membership-style filtering inside a list comprehension, all working together to turn a raw list of numbers into a small business report.

20.2 Student Grading

```
def get_grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'

students = {'Ada': 95, 'Grace': 82, 'Alan': 58}
for name, score in students.items():
    passed = score >= 60
    print(f'{name}: {get_grade(score)} (Passed: {passed})')
```

Comparison operators chained through elif branches classify a numeric score into a letter grade, while a separate boolean comparison (score >= 60) independently answers a related but distinct pass/fail question, showing how the same underlying value can be evaluated by different operator-driven conditions for different purposes.

20.3 Financial Calculations

```
principal = 5000
annual_rate = 0.06
years = 10

compound_amount = principal * (1 + annual_rate) ** years
interest_earned = compound_amount - principal
```

```
print(f'Final amount: {compound_amount:.2f}')
print(f'Interest earned: {interest_earned:.2f}')
```

The compound interest formula directly demonstrates operator precedence in action (section 10): Python computes (1 + annual_rate) inside the parentheses first, raises it to the power of years next (since ** outranks *), and only then multiplies by principal, exactly matching the mathematical formula's intended order.

20.4 Age Verification

```
def can_access_content(age, has_parental_consent):
    if age >= 18:
        return True
    return age >= 13 and has_parental_consent

print(can_access_content(15, True))
print(can_access_content(15, False))
print(can_access_content(10, True))
```

Combines comparison and logical operators to encode a real access-control business rule: adults are always allowed, minors 13 and older are allowed only with consent, and anyone younger is never allowed regardless of consent, since the function returns True from the first branch or falls through to a single combined and condition.

20.5 Data Validation

```
def validate_record(record):
    errors = []
    if 'email' not in record or not record['email']:
        errors.append('Missing email')
    if 'age' in record and not (0 < record['age'] < 120):
        errors.append('Invalid age')
    return errors

print(validate_record({'email': '', 'age': 200}))
print(validate_record({'email': 'ada@example.com', 'age': 36}))
```

Code Line	Explanation
`'email' not in record or not record['email']`	Combines membership (checking the key exists) with truthiness (checking the value is not an empty string) using or and not, exploiting short-circuit evaluation from section 11 so the second check only matters if the key is actually present.
`'age' in record and not (0 < record['age'] < 120)`	Combines membership, a chained comparison (section 19), and logical not into one validation rule: only check the age range if an age was actually provided at all.

20.6 Filtering Data

```
transactions = [
    {'amount': 250, 'type': 'credit'},
    {'amount': -80, 'type': 'debit'},
    {'amount': 500, 'type': 'credit'},
```

```
    {'amount': -20, 'type': 'debit'},
]

large_credits = [t for t in transactions if t['type'] == 'credit' and t['amount'] >
200]
print(large_credits)
```

A list comprehension driven entirely by comparison and logical operators filters a list of dictionaries down to only the records matching a compound business condition, a pattern that appears constantly in real data processing code.

20.7 Risk Classification (AI / Data Science Style Condition)

```
def classify_risk(credit_score, debt_to_income):
    if credit_score >= 750 and debt_to_income < 0.3:
        return 'Low Risk'
    elif credit_score >= 650 and debt_to_income < 0.5:
        return 'Medium Risk'
    else:
        return 'High Risk'

applicants = [(780, 0.2), (680, 0.45), (600, 0.6)]
for score, dti in applicants:
    print(classify_risk(score, dti))
```

This mirrors a simplified version of a real business rule used in lending and credit risk systems: multiple numeric thresholds, combined with and, are checked in priority order through elif, exactly the kind of rule-based classification logic that often sits alongside, or as a baseline before, a more sophisticated statistical or machine learning model.

21. Practice Exercises

Work through each exercise before checking the answer underneath. Exercises are grouped into Beginner, Intermediate and Advanced tiers, and every answer includes a full explanation, not just the final value.

21.1 Beginner Exercises

Exercise B1

What does the following code print?

```
print(17 % 5)
```

Answer: Answer: 2. Explanation: 17 divided by 5 is 3 with a remainder of 2 (5 * 3 = 15, and 17 - 15 = 2); % returns exactly that remainder.

Exercise B2

What does the following code print?

```
x = 10
y = '10'
print(x == y)
```

Answer: Answer: False. Explanation: x is an integer and y is a string; == between an int and a str is always False, since Python never implicitly converts between numeric and text types for equality comparison.

Exercise B3

What does the following code print?

```
a = True
b = False
print(a and b)
print(a or b)
```

Answer: Answer: False, then True. Explanation: and requires both operands to be truthy, and b is False, so the first line is False. or only needs one truthy operand, and a is True, so the second line is True.

Exercise B4

Fix the bug in this code, which is supposed to check whether count equals 5.

```
count = 5
if count = 5:
    print('Count is five')
```

Answer: Answer: Change if count = 5: to if count == 5:. A single = is assignment, not comparison; using it inside an if condition is actually a SyntaxError in Python, which is Python's built-in protection against this exact common mistake.

Exercise B5

What does the following code print?

```
print(bool(0))
print(bool(''))
print(bool([0]))
print(bool('False'))
```

Answer: Answer: False, False, True, True. Explanation: 0 and '' are both falsy. [0] is truthy because it is a non-empty list, even though its only element is falsy; truthiness of a container depends on whether it has elements, not on their contents. 'False' is a non-empty string (four characters), so it is truthy, even though it reads like the word 'False'.

21.2 Intermediate Exercises

Exercise I1

Without running it, determine what this expression evaluates to, and explain the order of operations.

```
print(2 + 3 * 4 ** 2 - 1)
```

Answer: Answer: 49. Explanation: exponent first, 4 ** 2 = 16; then multiplication, 3 * 16 = 48; then left to right addition and subtraction, 2 + 48 = 50, then 50 - 1 = 49.

Exercise I2

What does the following code print, and why?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Answer: Answer: [1, 2, 3, 4]. Explanation: b = a does not copy the list; it makes b another name pointing at the exact same list object as a (confirmed by a is b being True). Mutating the list through b is therefore visible through a as well, since there is really only one list involved.

Exercise I3

Explain why this code raises an error, and rewrite it so it does not.

```
scores = []
average = sum(scores) / len(scores)
```

Answer: Answer: ZeroDivisionError, since len(scores) is 0 and division by zero is undefined. Fix: average = sum(scores) / len(scores) if scores else 0, using short-circuit-style conditional logic to avoid ever dividing by zero.

Exercise I4

What does the following code print? Explain using short-circuit evaluation.

```
def noisy(label, value):
```

```
    print(f'Evaluating {label}')
    return value

result = noisy('A', False) and noisy('B', True)
print(result)
```

Answer: Answer: prints 'Evaluating A', then False. Explanation: noisy('A', False) is called and returns False; since and already knows the whole expression must be falsy once it sees a falsy left operand, it short-circuits and never calls noisy('B', True) at all, so 'Evaluating B' is never printed, and result is simply the value False that and returned.

Exercise I5

What does the following code print?

```
x = 5
print(1 < x < 3 or x > 4)
```

Answer: Answer: True. Explanation: the chained comparison 1 < x < 3 means (1 < 5) and (5 < 3), which is True and False, giving False overall. Then False or (x > 4) is evaluated; x > 4 is 5 > 4, which is True, so the whole or expression is True.

21.3 Advanced Exercises

Exercise A1

Explain precisely what each line prints and why, paying close attention to CPython's integer caching behavior.

```
a = 100
b = 100
print(a is b)

c = 1000
d = 1000
print(c is d)
```

Answer: Answer: True, then typically False (though this is not guaranteed by the language). Explanation: CPython caches small integers roughly from -5 to 256 as shared singleton objects, so a and b, both 100, point to the same cached object, making is True. 1000 falls outside that cached range, so c and d are ordinarily two separate integer objects with equal value but different identity, making is typically False. This is precisely why is should never be used for numeric value comparisons; == is always the correct, reliable choice.

Exercise A2

Implement a Fraction class where + correctly adds two fractions using cross-multiplication, and demonstrate it.

```
class Fraction:
    def __init__(self, numerator, denominator):
        self.numerator = numerator
        self.denominator = denominator

    def __add__(self, other):
```

```
        new_num = self.numerator * other.denominator + other.numerator *
self.denominator
        new_den = self.denominator * other.denominator
        return Fraction(new_num, new_den)

    def __repr__(self):
        return f'{self.numerator}/{self.denominator}'

result = Fraction(1, 2) + Fraction(1, 3)
print(result)
```

Answer: Answer: 5/6. Explanation: __add__ implements the cross-multiplication rule for adding fractions with different denominators: (1*3 + 1*2) / (2*3) = (3 + 2) / 6 = 5/6. The __repr__ method controls how the resulting Fraction object is displayed when printed.

Exercise A3

Explain the output of this bitwise flags example.

```
ADMIN = 0b001
EDITOR = 0b010
VIEWER = 0b100

user_roles = ADMIN | VIEWER
print(bool(user_roles & EDITOR))
print(bool(user_roles & ADMIN))
```

Answer: Answer: False, then True. Explanation: user_roles combines ADMIN (001) and VIEWER (100) with |, producing 101. Checking user_roles & EDITOR isolates the EDITOR bit (010); since that bit is not set in 101, the AND result is 0, which is falsy. Checking user_roles & ADMIN isolates the ADMIN bit (001), which is set, producing a non-zero (truthy) result.

Exercise A4

Why does this code raise a TypeError, and what does it reveal about how Python resolves operators for custom classes?

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)
```

Answer: Answer: TypeError: unsupported operand type(s) for +: 'Point' and 'Point'. Explanation: Point never defines __add__, so Python has no instructions for what + should do between two Point instances; unlike built-in types, a custom class gets no default arithmetic behavior at all beyond identity-based equality. The fix, as shown in section 14, is to explicitly define __add__(self, other) on the class.

Exercise A5

Predict the output, and explain the role of operator precedence and the walrus-free chained logic involved.

```
x = 5
y = 0
print(x > 0 and y != 0 and x / y > 1)
```

Answer: Answer: False, and critically, no ZeroDivisionError is raised. Explanation: and evaluates left to right and short-circuits the moment it hits a falsy operand. x > 0 is True, so evaluation continues; y != 0 is False (y is 0), so and immediately short-circuits and returns False without ever evaluating x / y > 1, which would otherwise raise a ZeroDivisionError. This is the exact guard-condition pattern from section 11.4.1 preventing an error before it can happen.

22. Quick-Reference Cheat Sheet

22.1 Arithmetic Operators

Code Line	Explanation
`+`	Addition / concatenation / list merge. Example: 3 + 4 -> 7.
`-`	Subtraction / set difference. Example: 10 - 4 -> 6.
`*`	Multiplication / repetition. Example: 'ab' * 2 -> 'abab'.
`/`	True division, always returns float. Example: 7 / 2 -> 3.5.
`//`	Floor division, rounds toward negative infinity. Example: -7 // 2 -> -4.
`%`	Modulus (remainder). Example: 7 % 2 -> 1.
`**`	Exponentiation, right-associative. Example: 2 ** 10 -> 1024.

22.2 Comparison Operators

Code Line	Explanation
`==, !=`	Equal to / not equal to. Compares value, works across compatible types (e.g. int and float).
`>, <, >=, <=`	Ordering comparisons. Works on numbers, strings, lists, tuples (lexicographic); not defined for dict.

22.3 Assignment Operators

Code Line	Explanation
`=`	Basic assignment / binding.
`+=, -=, *=, /=, //=, %=, **=`	Augmented arithmetic assignment: x += 1 is short for x = x + 1.
`&=, |=, ^=, >>=, <<=`	Augmented bitwise assignment.

22.4 Logical Operators

Code Line	Explanation
`and`	True only if both operands truthy; short-circuits on the first falsy operand.
`or`	True if at least one operand truthy; short-circuits on the first truthy operand.
`not`	Inverts truthiness; always returns a plain bool.

22.5 Identity and Membership Operators

Code Line	Explanation
`is, is not`	Object identity (same object in memory). Use for None, True, False; never for numbers or strings.
`in, not in`	Membership test: substring (str), element (list/tuple/set), key (dict by default).

22.6 Bitwise Operators

Code Line	Explanation
`&`	Bitwise AND. Also set intersection.

Code Line	Explanation
`|`	Bitwise OR. Also set union / dict merge.
`^`	Bitwise XOR. Also set symmetric difference.
`~`	Bitwise NOT (unary): ~x equals -(x + 1).
`<<, >>`	Left / right shift: equivalent to multiplying / floor-dividing by 2**n.

22.7 Precedence Quick Order (Highest to Lowest)

() -> ** -> unary +/-/~ -> * / // % -> binary +/- -> << >> -> & -> ^ -> | -> comparisons/is/in -> not -> and -> or -> conditional (if/else) -> assignment.

22.8 Falsy Values (Everything Else Is Truthy)

None, False, 0, 0.0, 0j, '', [], (), {}, set(), range(0).

22.9 Overloadable Dunder Methods

Code Line	Explanation
`__add__, __sub__, __mul__, __truediv__, __floordiv__, __mod__, __pow__`	Arithmetic operators.
`__eq__, __ne__, __lt__, __le__, __gt__, __ge__`	Comparison operators.
`__and__, __or__, __xor__, __invert__, __lshift__, __rshift__`	Bitwise operators.
`__contains__, __getitem__`	Membership and indexing.

23. Conclusion

Operators are the smallest, most frequently used pieces of syntax in Python, yet they carry an enormous amount of hidden structure: precedence rules that determine evaluation order, associativity rules that resolve ties between operators of equal rank, short-circuit behavior that can prevent errors and save computation, and a dispatch mechanism through dunder methods that lets the exact same symbol mean something entirely different depending on the types involved.

Understanding operators at this depth, not just what each one does but why Python evaluates it the way it does, is what separates writing code that happens to work from writing code whose behavior you can predict with confidence in every case, including the edge cases: negative floor division, chained comparisons, truthy empty containers, and the subtle difference between is and ==.

The practical exercises and real-world examples throughout this handbook were chosen deliberately to connect these rules to the kind of conditions, filters, and calculations that appear constantly in everyday Python code, from a simple grading script to a credit risk classifier. Returning to the precedence table in section 10 and the cheat sheet in section 22 as quick references, while keeping the underlying reasoning from each section in mind, is the most direct path to genuine fluency with Python operators.

End of Document

Kajola Gbenga | Data Scientist & AI Engineer